

Inquisitors AI Assistant

Final Project Report

Project: Inquisitors AI Assistant Chatbot Team Member: [Your Name] Teammate: [Teammate Name] GitHub Repository: https://github.com/SUNBALSHEHZADI/Inquisitors_AI_chatbot Date: 20 August 2026

> Replace the bracketed identity fields before submission. This Markdown file is the editable report source. final-project-report.docx is the editable formatted version and final-project-report.pdf is the submission copy.

Table of Contents

1. Abstract
2. Introduction
3. Project Objectives
4. Scope
5. Requirements Analysis
6. System Architecture
7. Knowledge Base
8. RAG Pipeline
9. Hallucination & Fallback Strategy
10. Technology Stack
11. Project Structure
12. API Documentation
13. Frontend Prototype
14. Implementation Results
15. Testing Plan
16. Limitations
17. Future Enhancements
18. Conclusion

1. Abstract

The Inquisitors AI Assistant is a browser-based educational chatbot for the Inquisitors Society. It combines a FastAPI backend, FAISS vector search, Sentence Transformers embeddings, a Groq large language model, and SQLite session memory. The system answers verified society questions and can also teach AI, Machine Learning, Data Science, Python, statistics, algorithms, and related concepts. The project follows the Learn, Grow, Build vision by combining trustworthy information retrieval with practical learning guidance.

2. Introduction

Students often need information about memberships, internships, events, departments, training, and contact channels. They also need accessible explanations of technical concepts and guidance for learning projects. The

Inquisitors AI Assistant provides one conversational interface for both needs. Official society facts are grounded in the project knowledge base, while general academic questions are handled as educational tutoring. The frontend is served by FastAPI so the local application can run from one URL.

3. Project Objectives

- Provide clear and student-friendly answers about Inquisitors Society.
- Explain AI, ML, Data Science, programming, and research concepts.
- Ground official claims in verified Markdown knowledge.
- Retrieve relevant information using semantic vector search.
- Maintain conversation history using session-based SQLite storage.
- Provide a professional responsive interface with suggested questions.
- Provide fallback behavior when official information is unavailable.
- Demonstrate a practical RAG application that supports Learn, Grow, Build.

4. Scope

In Scope

The system covers society introduction, departments, membership, internships, events, training, services, FAQs, contact information, official social channels, and an educational curriculum for AI, ML, and Data Science. Users can ask natural-language questions, continue conversations, use suggested questions, use browser voice input where supported, and request human support information.

Out of Scope

The system does not replace official administration, create application registrations, guarantee current schedules or certificates, provide authentication, or permanently store production-grade user records. Current dates, fees, policies, and registration links must be confirmed through official channels.

5. Requirements Analysis

Functional Requirements

- Accept natural-language messages up to 4000 characters.
- Validate requests and session IDs with Pydantic.
- Retrieve relevant knowledge chunks using FAISS.
- Generate responses through Groq.
- Return answers, source names, and session IDs as JSON.
- Store and retrieve conversation history.
- Explain educational concepts when a question is instructional.
- Return a clear fallback for unsupported official questions.
- Serve the frontend and static assets from FastAPI.

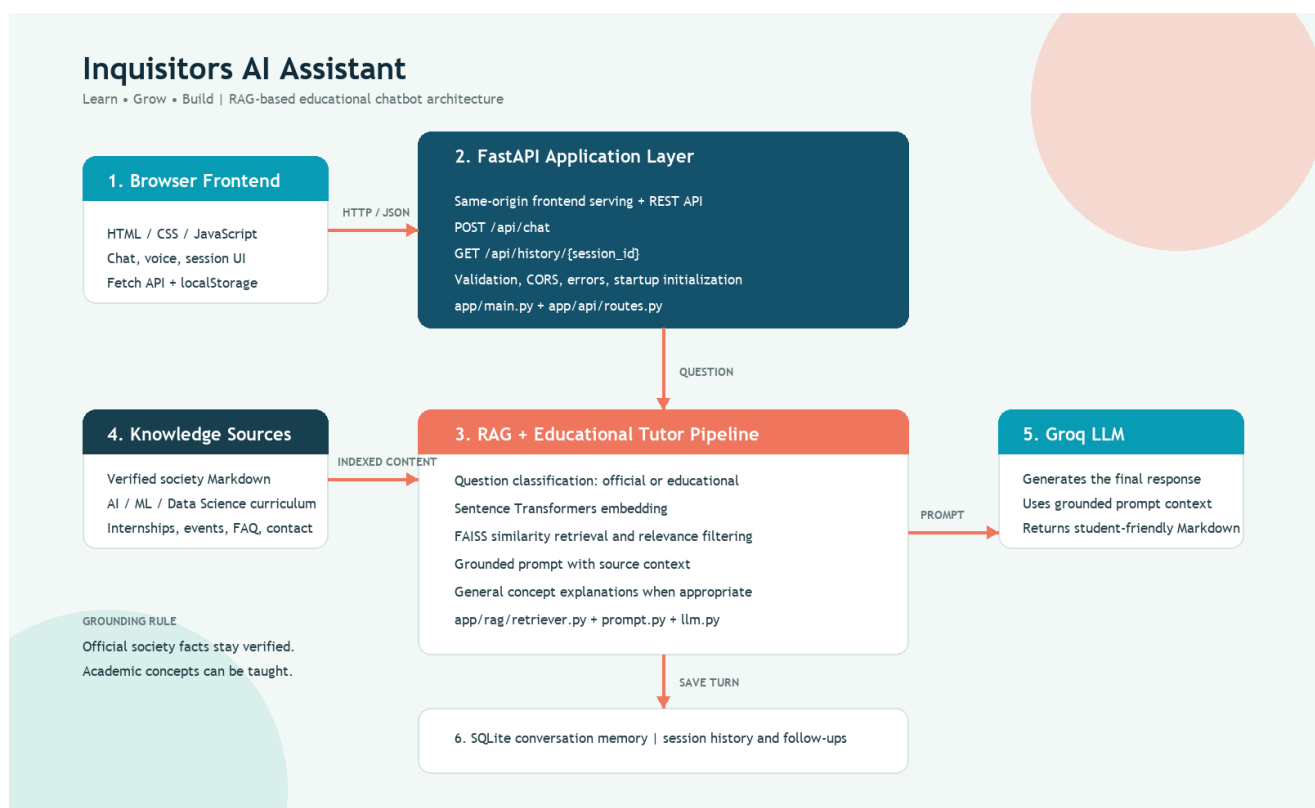
Non-Functional Requirements

- Responsive browser interface.
- Professional, student-friendly tone.
- No API keys in source code.
- Safe HTML escaping before displaying model output.
- Graceful error handling.
- Local startup target of approximately 2 GB RAM and 500 MB disk.

6. System Architecture

The system follows a layered architecture: browser frontend, FastAPI application, RAG and educational tutor pipeline, knowledge sources, Groq response generation, and SQLite conversation memory.

Inquisitors AI Assistant system architecture



Architecture image: screenshots/system-architecture.png Editable vector source: screenshots/system-architecture.svg

Request Flow

1. The user enters a question in the browser.
2. The frontend sends JSON to POST /api/chat.
3. FastAPI validates the message and session.
4. The retriever embeds the query and searches FAISS.
5. The prompt builder creates an answer context.
6. The Groq model generates a response.
7. User and assistant messages are saved in SQLite.

8. The API returns the answer and source names to the browser.

7. Knowledge Base

The knowledge base is stored in `knowledge_base/processed/` as structured Markdown. It includes society, departments, events, FAQ, internships, membership, services, social media, sources, training, contact information, and `learning_curriculum.md`.

The curriculum document extends the platform beyond an information FAQ. It covers AI foundations, the Machine Learning learning path, Data Science workflow, evaluation metrics, beginner projects, study methods, and scope boundaries. The vector store was rebuilt with 59 chunks so the new curriculum is searchable.

8. RAG Pipeline

Retrieval-Augmented Generation combines semantic retrieval with language generation:

1. Clean and validate the question.
2. Include limited recent history for follow-up retrieval.
3. Encode the question with `all-MiniLM-L6-v2`.
4. Search the FAISS `IndexFlatL2` vector index.
5. Check the best distance against the relevance threshold.
6. Build a prompt containing source names and retrieved text.
7. Ask Groq to generate a concise response.
8. Return sources and save the conversation.

Educational questions are allowed to receive general concept explanations when the society knowledge base does not contain the requested concept. Society-specific questions remain restricted to verified context.

9. Hallucination & Fallback Strategy

The assistant uses a two-mode policy. For official questions about Inquisitors Society, the model must use retrieved verified context and must not invent dates, fees, links, names, policies, schedules, or program promises. If relevant context is unavailable, the API returns a clear fallback message directing the user to official administration channels.

For educational questions, the assistant may explain general academic concepts. It must label general guidance as educational knowledge and must not turn it into an official Inquisitors course promise. This balance supports learning while preserving trust in society information.

10. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, vanilla JavaScript, Fetch API
Backend	Python, FastAPI, Uvicorn
Validation	Pydantic
Retrieval	FAISS CPU, Sentence Transformers
Embeddings	<code>all-MiniLM-L6-v2</code>
LLM	Groq API, <code>openai/gpt-oss-20b</code>

Layer	Technology
Memory	SQLite
Data format	Markdown, JSON, pickle metadata
Testing	pytest, manual scenario CSV
Local serving	FastAPI same-origin frontend and API

11. Project Structure

```

inquisitors_chatbot/
|- app/ FastAPI and RAG application
|  |- main.py Application and frontend serving
|  |- api/ Chat and history routes
|  |- rag/ Retrieval, prompts, LLM, memory
|- frontend/ HTML, CSS, JavaScript, logo assets
|- knowledge_base/processed/ Verified society and curriculum Markdown
|- vector_store/ FAISS index and chunk metadata
|- data/ SQLite conversation databases
|- test/ Automated tests and manual scenarios
|- docs/ Technical and user documentation
|- screenshots/ UI and architecture images
|- Dockerfile Container deployment configuration
|- requirements.txt Python dependencies

```

12. API Documentation

POST /api/chat

Request:

```

{
  "message": "Explain overfitting in machine learning",
  "session_id": "web-demo-session"
}

```

Response:

```

{
  "answer": "...",
  "sources": ["learning_curriculum.md"],
  "session_id": "web-demo-session"
}

```

GET /api/history/{session_id}

Returns the saved conversation messages for a session.

DELETE /api/history/{session_id}

Clears the selected session history.

GET /health

Returns service readiness, database type, LLM provider, and application version.

GET /api-info

Returns basic application metadata and architecture information.

13. Frontend Prototype

The frontend is a responsive single-page interface with an editorial educational visual style. The hero includes a 3D orbit composition with the original Inquisitors logo and relevant student-learning imagery. The chatbot includes an assistant header, suggested questions, message bubbles, voice input, read-aloud controls, source display, loading states, support escalation, and a floating Ask AI button.

The frontend is served from FastAPI at <http://127.0.0.1:8000>, which avoids the earlier split-origin problem between the static server and API. The original logo at `frontend/assets/logo.png` is preserved.

14. Implementation Results

- FastAPI backend initializes FAISS, embeddings, Groq, and SQLite successfully.
- The root endpoint serves the frontend.
- CSS, JavaScript, and logo routes return successfully from the same origin.
- Educational questions are answered with tutoring guidance.
- Official society questions remain grounded and source-aware.
- The FAISS store contains 59 indexed chunks.
- Architecture documentation includes PNG and editable SVG diagrams.
- The latest implementation is available on GitHub at:

https://github.com/SUNBALSHEHZADI/Inquisitors_AI_chatbot

15. Testing Plan

Automated Tests

The focused RAG test suite covers context formatting, source uniqueness, relevance threshold behavior, follow-up search context, educational question detection, and prompt policy. The latest focused result is 6 passed.

Manual Demonstration Tests

The project includes 40 documented scenarios covering normal queries, FAQs, contact, services, complex questions, incomplete input, misspellings, unrelated questions, repeated queries, out-of-scope requests, invalid input, long messages, multi-question requests, sessions, edge cases, grounding, accuracy, fallback, errors, and UX tone.

Suggested Demo Questions

- What is Inquisitors Society?
- Explain machine learning for beginners.
- What is overfitting?
- Create a Data Science roadmap.
- What internship domains are available?
- How can I become a member?

16. Limitations

- Groq API access and a valid GROQ_API_KEY are required for live answers.
- Sentence Transformers and FAISS require more memory than a simple static site.

- SQLite is not appropriate for durable multi-instance production storage.
- The knowledge base requires manual updates when official information changes.
- Educational explanations can be broad and should not be treated as formal academic certification.
- Free hosting platforms may sleep, restart, or limit memory.
- The application currently supports English as its primary language.

17. Future Enhancements

- Add authentication and role-based administration.
- Add an admin interface for knowledge-base updates and re-indexing.
- Add multilingual responses and accessibility improvements.
- Replace SQLite with PostgreSQL or another persistent hosted database.
- Add analytics with privacy controls.
- Add automated source freshness checks.
- Add streaming responses and richer code examples.
- Add a formal course catalog with instructor-approved materials.
- Add deployment monitoring, rate limiting, and usage budgets.

18. Conclusion

The Inquisitors AI Assistant is a complete working prototype that combines a professional frontend with a grounded RAG backend. It answers official society questions with verified context and supports the wider Learn, Grow, Build mission by teaching technical concepts and suggesting practical learning paths. The project includes documented architecture, a searchable curriculum, conversation memory, fallback safeguards, automated tests, and report assets. It is ready for local demonstration, evaluation, and future production hardening.